Bayesian Kelly Criterion: From Point Estimate to Probability Distribution

Four scenarios showing why treating your edge as uncertain produces better position sizing

AUTHOR
Patrick Lefler

PUBLISHED
March 17, 2026

ABSTRACT

The Kelly Criterion holds a unique spot in finance. It is often cited, misused, and rarely tested against its own assumptions. The formula works best when the win probability is known exactly. In reality, win probabilities come from limited historical data, which can be noisy. For example, a strategy that won 58% of its last 12 trades might really have a true win rate anywhere from 40% to 75%. The standard Kelly calculation ignores this uncertainty. It turns a noisy estimate into a confident position size.

This project says that direct translation risks Kelly-based sizing. It can lead to serious, unmeasured issues. The proposed method treats win probability as a Beta-distributed random variable, not a fixed number. Wins and losses update a prior distribution using Bayes' theorem. This produces a posterior that shows both the evidence and remaining uncertainty. By applying the Kelly formula to 10,000 draws from this posterior, we get a range of recommended stakes. This range is broad with weak evidence and narrow with strong evidence. This results in a cautious, percentile-based policy.

The 25th percentile of this distribution, called p25, is the suggested default stake. It's optimal if the true win rate falls within the upper three-quarters of the uncertainty range. Four scenarios demonstrate the practical effects of this framework. With just 12 observations, the posterior is too broad. It can't confidently support a 16% stake based on a point estimate. Kelly responds with a near-zero size at the conservative percentile.

The second and third scenarios show that as more evidence comes in, the p25 recommendation moves closer to full Kelly. This illustrates the framework's self-correcting nature. The fourth scenario is crucial. It uses the same inputs as the first but simulates outcomes at a true win rate of 52%, rather than the posterior mean of 56%. This small difference highlights how in-sample models can overstate out-of-sample performance. Full Kelly is heavily punished, while the p25 stake remains intact. The overconfidence premium is the gap between median and conservative Kelly recommendations. This gap helps risk managers and investment committees see how reliable an edge estimate is.

Setup

▼ Show code

```r
library(plotly)
library(dplyr)
library(tidyr)
library(ggplot2)
library(knitr)
library(kableExtra)
library(scales)
library(htmltools)
library(sessioninfo)

# ── sandstone brand palette ──────────────────────────────────────────────────
plot_background      <- "#FEFEFA"
plot_blacktext       <- "#2C2C2C"
plot_greytext        <- "#707073"
plot_bluetext        <- "#000066"
plot_redtext         <- "#800000"
plot_fill_beige      <- "#FFE2C3"
plot_fill_blue       <- "#447099"
plot_fill_brightblue <- "#0A0AFF"
plot_fill_crimson    <- "#B94A48"
plot_fill_lightgrey  <- "#E8E8E8"
plot_fill_lightblue  <- "#BDD7E7"
plot_fill_red        <- "#DC143C"

blue_shade_1  <- "#EFF3FF"; blue_shade_2  <- "#BDD7E7"
blue_shade_3  <- "#6BAED6"; blue_shade_4  <- "#2171B5"

green_shade_1 <- "#EDF8E9"; green_shade_2 <- "#BAE4B3"
green_shade_3 <- "#74C476"; green_shade_4 <- "#238B45"

grey_shade_1  <- "#F7F7F7"; grey_shade_2  <- "#CCCCCC"
grey_shade_3  <- "#969696"; grey_shade_4  <- "#525252"

red_shade_1   <- "#FEE5D9"; red_shade_2   <- "#FCAE91"
red_shade_3   <- "#FB6A4A"; red_shade_4   <- "#CB181D"

yellow_blue_shade_1 <- "#FFFFCC"; yellow_blue_shade_2 <- "#C7E9B4"
yellow_blue_shade_3 <- "#7FCDBB"; yellow_blue_shade_4 <- "#41B6C4"
yellow_blue_shade_5 <- "#1D91C0"; yellow_blue_shade_6 <- "#225EA8"
yellow_blue_shade_7 <- "#0C2C84"

# ── Shared ggplot theme ──────────────────────────────────────────────────────
theme_kelly <- theme_minimal(base_family = "Roboto") +
  theme(
    plot.background  = element_rect(fill = plot_background, color = NA),
    panel.background = element_rect(fill = plot_background, color = NA),
    panel.grid.major = element_line(color = plot_fill_lightgrey, linewidth = 0.4),
    panel.grid.minor = element_blank(),
    text             = element_text(color = plot_blacktext),
    plot.title       = element_text(size = 13, face = "bold"),
    plot.subtitle    = element_text(size = 11, color = plot_greytext),
    axis.text        = element_text(size = 10, color = plot_greytext),
    axis.title       = element_text(size = 11, color = plot_blacktext),
    legend.position  = "bottom",
    legend.text      = element_text(size = 10),
    plot.caption     = element_text(size = 9, color = plot_greytext, hjust = 0)
  )

# ── Scenario definitions ─────────────────────────────────────────────────────
```

```r
scenarios <- list(
  s1 = list(
    label       = "Scenario 1",
    title       = "The Danger Zone",
    subtitle    = "Sparse data, wide posterior — the point estimate misleads",
    alpha_prior = 2, beta_prior = 2,
    wins = 7, n_trials = 12,
    b_odds = 2.0,
    true_p = NULL
  ),
  s2 = list(
    label       = "Scenario 2",
    title       = "The Transition",
    subtitle    = "Moderate data — p25 turns meaningfully positive",
    alpha_prior = 3, beta_prior = 2,
    wins = 15, n_trials = 25,
    b_odds = 2.0,
    true_p = NULL
  ),
  s3 = list(
    label       = "Scenario 3",
    title       = "The Experienced Operator",
    subtitle    = "Rich data — p25 converges toward full Kelly",
    alpha_prior = 4, beta_prior = 2,
    wins = 32, n_trials = 50,
    b_odds = 2.0,
    true_p = NULL
  ),
  s4 = list(
    label       = "Scenario 4",
    title       = "Optimism Meets Reality",
    subtitle    = "Identical inputs to Scenario 1 — but the true edge is lower than estimated",
    alpha_prior = 2, beta_prior = 2,
    wins = 7, n_trials = 12,
    b_odds = 2.0,
    true_p = 0.52
  )
)

# ── Core helper functions ──────────────────────────────────────────────
posterior_params <- function(sc) {
  list(
    alpha_post = sc$alpha_prior + sc$wins,
    beta_post  = sc$beta_prior  + (sc$n_trials - sc$wins)
  )
}

kelly_fraction <- function(p, b_decimal) {
  b_net <- b_decimal - 1
  pmax((b_net * p - (1 - p)) / b_net, 0)
}

simulate_path <- function(p_draw, b_decimal, n, f_pct) {
  b_net <- b_decimal - 1
  bk    <- numeric(n + 1)
  bk[1] <- 100000
  for (i in seq_len(n)) {
    win     <- rbinom(1, 1, p_draw)
    bk[i+1] <- bk[i] * (1 + f_pct * ifelse(win == 1, b_net, -1))
    if (bk[i+1] <= 0) { bk[(i+1):(n+1)] <- 0; break }
  }
  bk
}

# ── Plot 1: Prior / Likelihood / Posterior density overlay ──────────────
make_density_plot <- function(sc) {
  pp    <- posterior_params(sc)
  p_seq <- seq(0.001, 0.999, length.out = 500)

  prior_y     <- dbeta(p_seq, sc$alpha_prior, sc$beta_prior)
  posterior_y <- dbeta(p_seq, pp$alpha_post,  pp$beta_post)
  lik_raw     <- dbinom(sc$wins, sc$n_trials, p_seq)
  lik_y       <- lik_raw / max(lik_raw) * max(posterior_y)
  post_mean   <- pp$alpha_post / (pp$alpha_post + pp$beta_post)

  df <- bind_rows(
    data.frame(p = p_seq, density = prior_y,     curve = "Prior"),
    data.frame(p = p_seq, density = lik_y,        curve = "Likelihood (scaled)"),
    data.frame(p = p_seq, density = posterior_y,  curve = "Posterior")
  ) |>
    mutate(curve = factor(curve,
                          levels = c("Prior", "Likelihood (scaled)", "Posterior")))

  g <- ggplot(df, aes(x = p, y = density, color = curve)) +
    geom_line(aes(linetype = curve), linewidth = 0.9) +
    geom_area(
      data = filter(df, curve == "Posterior"),
      aes(fill = curve), alpha = 0.15, show.legend = FALSE
    ) +
    scale_color_manual(values = c(
      "Prior"               = grey_shade_4,
      "Likelihood (scaled)" = grey_shade_3,
      "Posterior"           = blue_shade_4
    )) +
    scale_fill_manual(values = c("Posterior" = blue_shade_2)) +
    scale_linetype_manual(values = c(
      "Prior"               = "dotted",
      "Likelihood (scaled)" = "dashed",
      "Posterior"           = "solid"
    )) +
    scale_x_continuous(labels = percent_format(accuracy = 1), limits = c(0, 1)) +
    labs(x = "Win probability p", y = "Density", color = NULL, linetype = NULL) +
```

```r
    theme_kelly

  ggplotly(g) |>
    layout(
      paper_bgcolor = plot_background,
      plot_bgcolor  = plot_background,
      shapes = list(list(
        type = "line", x0 = post_mean, x1 = post_mean, y0 = 0, y1 = 1,
        yref = "paper",
        line = list(color = blue_shade_4, width = 1.5, dash = "dot")
      )),
      annotations = list(list(
        x = post_mean + 0.01, y = 0.95, yref = "paper", xanchor = "left",
        text = sprintf("posterior mean<br>= %.3f", post_mean),
        showarrow = FALSE,
        font = list(size = 11, color = blue_shade_4, family = "Roboto")
      )),
      legend = list(orientation = "h", x = 0.5, xanchor = "center",
                    y = -0.2, font = list(size = 11))
    )
}

# ── Plot 2: Kelly fraction distribution ──────────────────────────────
make_kelly_plot <- function(sc, N = 10000, seed = 42) {
  set.seed(seed)
  pp    <- posterior_params(sc)
  b_net <- sc$b_odds - 1

  post_p  <- rbeta(N, pp$alpha_post, pp$beta_post)
  f_stars <- pmax((b_net * post_p - (1 - post_p)) / b_net, 0)

  sorted <- sort(f_stars)
  p10    <- sorted[floor(0.10 * N)]
  p25    <- sorted[floor(0.25 * N)]
  p50    <- sorted[floor(0.50 * N)]
  mean_f <- mean(f_stars)

  # Histogram bins
  hi     <- max(max(f_stars) * 1.05, 0.01)
  n_bins <- 55
  bw     <- hi / n_bins
  counts <- as.integer(tabulate(
    pmin(floor(f_stars / bw) + 1, n_bins), nbins = n_bins
  ))
  bin_x  <- (seq_len(n_bins) - 0.5) * bw
  ymax   <- max(counts) * 1.22

  pct_zero <- mean(f_stars == 0)

  vlines <- list(
    list(val = p10,    col = red_shade_4,    lbl = sprintf("p10 = %.1f%%", p10*100)),
    list(val = p25,    col = red_shade_3,    lbl = sprintf("p25 = %.1f%%", p25*100)),
    list(val = p50,    col = green_shade_4,  lbl = sprintf("p50 = %.1f%%", p50*100)),
    list(val = mean_f, col = plot_blacktext, lbl = sprintf("mean = %.1f%%", mean_f*100))
  )

  shapes <- lapply(vlines, function(v)
    list(type = "line", x0 = v$val, x1 = v$val, y0 = 0, y1 = ymax,
         line = list(color = v$col, width = 1.8,
                     dash = ifelse(v$val == mean_f, "dot", "solid")))
  )

  anns <- c(
    lapply(seq_along(vlines), function(i) {
      v <- vlines[[i]]
      list(x = v$val, y = ymax * (0.98 - (i-1)*0.10),
           xanchor = "left", text = paste0(" ", v$lbl),
           showarrow = FALSE,
           font = list(size = 11, color = v$col, family = "Roboto"))
    }),
    if (pct_zero > 0.05) list(list(
      x = 0.5, y = 1.02, xref = "paper", yref = "paper",
      xanchor = "center",
      text = sprintf("%.0f%% of draws recommend zero stake (edge below break-even)",
                     pct_zero * 100),
      showarrow = FALSE,
      font = list(size = 11, color = plot_greytext, family = "Roboto")
    ))
  )

  plot_ly() |>
    add_bars(
      x = bin_x, y = counts, width = bw * 0.88,
      marker = list(color = paste0(blue_shade_3, "bb"),
                    line  = list(color = blue_shade_4, width = 0.5)),
      hovertemplate = "f* = %{x:.3f}<br>count = %{y}<extra></extra>",
      showlegend = FALSE
    ) |>
    layout(
      paper_bgcolor = plot_background, plot_bgcolor = plot_background,
      font  = list(family = "Roboto", color = plot_blacktext),
      xaxis = list(title = "Recommended Kelly fraction f*", tickformat = ".0%",
                   gridcolor = plot_fill_lightgrey),
      yaxis = list(title = "Count (of 10,000 draws)",
                   gridcolor = plot_fill_lightgrey),
      shapes = shapes, annotations = anns,
      margin = list(t = 40, r = 20, b = 50, l = 70),
      bargap = 0.05
    )
}

# ── Sensitivity table data ───────────────────────────────────────────
```

```r
make_sensitivity_data <- function(sc, N_sim = 2000, n_bets = 200,
                                  bankroll = 100000, seed = 42) {
  set.seed(seed)
  pp <- posterior_params(sc)

  post_p   <- rbeta(N_sim, pp$alpha_post, pp$beta_post)
  f_draws  <- kelly_fraction(post_p, sc$b_odds)
  f_sorted <- sort(f_draws)

  pcts     <- c(p10 = 0.10, p25 = 0.25, p50 = 0.50, p75 = 0.75)
  f_values <- sapply(pcts, function(q) f_sorted[floor(q * N_sim)])
  f_values <- c(f_values, `Fixed 2%` = 0.02)

  results <- lapply(names(f_values), function(nm) {
    f <- f_values[nm]
    paths <- sapply(seq_len(N_sim), function(i) {
      p_draw <- if (!is.null(sc$true_p)) sc$true_p else post_p[i]
      simulate_path(p_draw, sc$b_odds, n_bets, f)
    })
    terminal  <- paths[n_bets + 1, ]
    drawdowns <- apply(paths, 2, function(p) {
      peak <- cummax(p)
      max((peak - p) / peak, na.rm = TRUE)
    })
    data.frame(
      Strategy               = nm,
      Stake                  = percent(f, accuracy = 0.1),
      `Bet ($100k)`          = dollar(bankroll * f, accuracy = 1),
      `Median final bankroll` = dollar(median(terminal), accuracy = 1),
      `Median drawdown`      = percent(median(drawdowns), accuracy = 0.1),
      `P(drawdown > 20%)`    = percent(mean(drawdowns > 0.20), accuracy = 0.1),
      check.names = FALSE
    )
  })

  list(table = do.call(rbind, results), f_values = f_values)
}

# ── Trajectory + terminal histogram ──────────────────────────────────────
make_trajectory_plots <- function(sc, f_values, N_paths = 200,
                                  n_bets = 500, bankroll = 100000, seed = 99) {
  set.seed(seed)
  pp <- posterior_params(sc)

  strat_names  <- c("Full Kelly (p50)", "Half Kelly",
                    "p25 (recommended)", "Fixed 2%")
  strat_colors <- c(red_shade_4, yellow_blue_shade_5,
                    blue_shade_4, grey_shade_4)
  strat_fracs  <- c(f_values["p50"], f_values["p50"] * 0.5,
                    f_values["p25"], 0.02)

  med_paths <- vector("list", 4)
  q25_paths <- vector("list", 4)
  q75_paths <- vector("list", 4)
  terminals <- vector("list", 4)

  for (i in seq_len(4)) {
    f   <- strat_fracs[i]
    mat <- sapply(seq_len(N_paths), function(j) {
      p_draw <- if (!is.null(sc$true_p)) {
        sc$true_p
      } else {
        rbeta(1, pp$alpha_post, pp$beta_post)
      }
      simulate_path(p_draw, sc$b_odds, n_bets, f)
    })
    med_paths[[i]] <- apply(mat, 1, median)
    q25_paths[[i]] <- apply(mat, 1, quantile, 0.25)
    q75_paths[[i]] <- apply(mat, 1, quantile, 0.75)
    terminals[[i]] <- pmax(mat[n_bets + 1, ], 1)
  }

  bet_seq <- 0:n_bets

  # ── Trajectory panel ──────────────────────────────────────────────────
  p1 <- plot_ly()
  for (i in seq_len(4)) {
    col <- strat_colors[i]
    med <- pmax(med_paths[[i]], 1)
    q25 <- pmax(q25_paths[[i]], 1)
    q75 <- pmax(q75_paths[[i]], 1)

    p1 <- p1 |> add_trace(
      x = c(bet_seq, rev(bet_seq)), y = c(q75, rev(q25)),
      type = "scatter", mode = "none", fill = "toself",
      fillcolor = paste0(col, "22"), showlegend = FALSE, hoverinfo = "skip"
    )
    p1 <- p1 |> add_trace(
      x = bet_seq, y = med, type = "scatter", mode = "lines",
      line = list(color = col, width = 2), name = strat_names[i],
      hovertemplate = paste0("<b>", strat_names[i], "</b><br>",
                             "Bet %{x}<br>$%{y:,.0f}<extra></extra>")
    )
  }

  p1 <- p1 |> layout(
    paper_bgcolor = plot_background, plot_bgcolor = plot_background,
    font  = list(family = "Roboto", color = plot_blacktext),
    xaxis = list(title = paste0("Bet number (n = ", n_bets, ")"),
                 gridcolor = plot_fill_lightgrey, range = c(0, n_bets)),
    yaxis = list(
      title     = "Bankroll ($)",
```

```
      type      = "log",
      tickvals  = c(1e3, 1e4, 1e5, 1e6, 1e7, 1e8, 1e9, 1e10, 1e11, 1e12),
      ticktext  = c("$1K", "$10K", "$100K", "$1M", "$10M",
                    "$100M", "$1B", "$10B", "$100B", "$1T"),
      gridcolor = plot_fill_lightgrey
    ),
    legend = list(orientation = "h", x = 0.5, xanchor = "center",
                  y = 1.12, yanchor = "top", font = list(size = 11)),
    margin    = list(t = 55, r = 20, b = 50, l = 90),
    hovermode = "x unified", height = 370
  )

  # —— Terminal histogram panel ————————————————————————————————————————
  all_term   <- unlist(terminals)
  log_lo     <- floor(min(log10(all_term)))
  log_hi     <- ceiling(max(log10(all_term)))
  log_breaks <- seq(log_lo, log_hi, length.out = 46)
  break_vals <- 10^log_breaks

  p2 <- plot_ly()
  for (i in seq_len(4)) {
    col      <- strat_colors[i]
    log_vals <- log10(terminals[[i]])
    h        <- hist(log_vals, breaks = log_breaks, plot = FALSE)
    mids     <- as.numeric(10^h$mids)
    counts   <- as.integer(h$counts)
    widths   <- as.numeric(diff(break_vals) * 0.85)

    p2 <- p2 |> add_bars(
      x = mids, y = counts, width = widths,
      name = strat_names[i], showlegend = TRUE,
      marker = list(color = paste0(col, "99"),
                    line  = list(color = col, width = 0.5)),
      opacity = 0.72,
      hovertemplate = paste0("<b>", strat_names[i], "</b><br>",
                             "Terminal: ~$%{x:,.0f}<br>",
                             "Count: %{y}<extra></extra>")
    )
  }

  p2 <- p2 |> layout(
    paper_bgcolor = plot_background, plot_bgcolor = plot_background,
    font = list(family = "Roboto", color = plot_blacktext),
    barmode = "overlay",
    xaxis = list(
      title    = "Terminal bankroll after 500 bets ($)",
      type     = "log",
      tickvals = c(1e3, 1e4, 1e5, 1e6, 1e7),
      ticktext = c("$1k", "$10k", "$100k", "$1M", "$10M"),
      gridcolor = plot_fill_lightgrey
    ),
    yaxis = list(title = paste0("Count (of ", N_paths, " runs)"),
                 gridcolor = plot_fill_lightgrey),
    shapes = list(list(
      type = "line", x0 = bankroll, x1 = bankroll, y0 = 0, y1 = 1,
      yref = "paper",
      line = list(color = plot_blacktext, width = 1.5, dash = "dot")
    )),
    annotations = list(list(
      x = bankroll, y = 0.96, yref = "paper", xanchor = "left",
      text = "  starting<br>  bankroll", showarrow = FALSE,
      font = list(size = 11, color = plot_blacktext, family = "Roboto")
    )),
    legend = list(orientation = "h", x = 0.5, xanchor = "center",
                  y = 1.12, yanchor = "top", font = list(size = 11)),
    margin = list(t = 55, r = 20, b = 60, l = 90), height = 300
  )

  browsable(tagList(
    as_widget(p1),
    tags$div(style = "height: 28px;"),
    as_widget(p2)
  ))
}
```

Introduction

The Kelly Criterion is a formula for position sizing. It was developed in 1956 by John L. Kelly Jr. at Bell Labs. Originally, it aimed to solve signal-to-noise issues in long-distance phone calls. Later, its use in gambling and investing became clear.

The formula helps you find the best fraction of capital to bet. It does this by maximizing the expected logarithm of wealth. This method outperforms others over long periods. In financial markets, gamblers and quantitative traders quickly adopted it. They found that the math for optimal betting works for securities with uneven payoffs too.

The Kelly Criterion is now used in:

- Quantitative trading
- Venture capital
- Sports betting
- Cryptocurrency management

It helps to convert an edge estimate into a position size. However, its main drawback is that it assumes the edge is known exactly. This project addresses that issue.

How to Understand this Project

Each scenario follows the same four-step structure. First, a prior/posterior chart shows how observed data updates our belief about the true win probability. Second, a Kelly fraction distribution histogram shows what the formula recommends across 10,000 draws from the posterior — a distribution rather than a single number. Third, a sensitivity table shows how each percentile stake performs over 200 bets. Fourth, wealth trajectory and terminal bankroll charts show the long-run consequences of each staking strategy over 500 bets.

How to interpret decimal odds

Decimal odds represent your total return for every $1 you stake — including getting your original dollar back.

- **Odds of 2.0** means if you bet $1 and win, you get $2 back — your $1 stake plus $1 profit. This is the equivalent of a fair coin flip in terms of payout structure. At these odds, the break-even win probability is exactly 50%.
- **Odds of 4.0** means if you bet $1 and win, you get $4 back — your $1 stake plus $3 profit. At these odds you only need to win more than 25% of the time to have a positive edge.

All four scenarios use odds of 2.0. The point estimate Kelly formula recommends staking `p - (1-p) = 2p - 1` of your bankroll — the fraction by which your win rate exceeds 50%.

Scenario 1: The Danger Zone

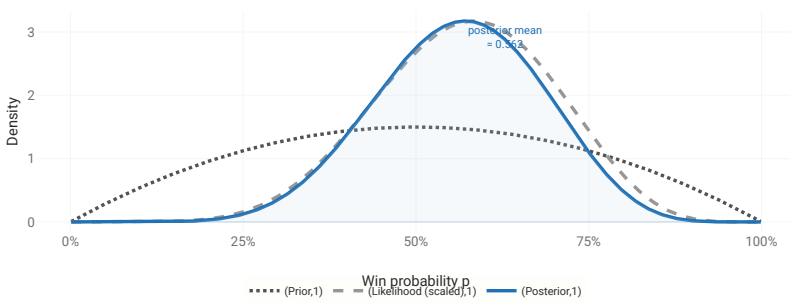
7 wins in 12 trials. Weakly informative prior Beta(2, 2). Decimal odds 2.0.

The point estimate win rate is 7/12 = 58.3%, which a traditional Kelly calculation translates directly into a confident 16.7% stake. But 12 observations is very little data. The posterior is wide enough that plausible values of the true win rate range from below 30% to above 80%. At these extremes, Kelly recommends either zero or aggressive over-staking. The distribution of recommended stakes reflects that uncertainty honestly.

Prior vs. posterior

▼ Show code

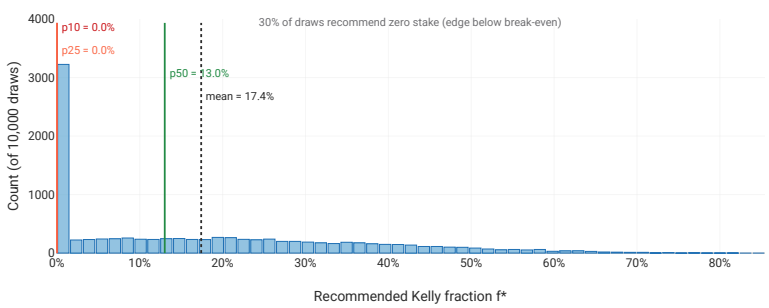
```
make_density_plot(scenarios$s1)
```



Kelly fraction distribution

▼ Show code

```
make_kelly_plot(scenarios$s1)
```



Staking policy

Understanding the sensitivity table (click to expand)

`Strategy` — The staking approach being tested. **p10**, **p25**, **p50**, and **p75** refer to percentiles of the Kelly fraction distribution — the stake recommended if your true win rate sits at that percentile of the uncertainty range. **Fixed 2%** is a flat baseline that stakes the same percentage every bet regardless of the estimated edge.

`Stake` — The fraction of your total bankroll placed on each bet, expressed as a percentage. A 5% stake on a $100,000 bankroll means $5,000 per bet.

`Bet ($100k)` — The dollar amount placed on each bet, assuming a $100,000 starting bankroll. This is simply the Stake percentage multiplied by $100,000.

`Median final bankroll` — The bankroll value at the end of 200 bets taken from the middle of 2,000 simulated runs — half the runs finished above this number and half finished below it. Think of it as the most typical outcome, not the best or worst case.

`Median drawdown` — The largest peak-to-trough decline in bankroll experienced during a typical run. A 30% median drawdown means the typical simulation saw the bankroll fall 30% from its highest point at some stage during the 200 bets. This measures how much pain you endure along the way, even if the bankroll ultimately recovers.

`P(drawdown > 20%)` — The proportion of simulated runs where the bankroll dropped more than 20% from its peak at some point. A value of 80% means 8 out of 10 simulations experienced at least one 20% drawdown. This is a measure of tail risk — how often things get uncomfortable even when the strategy has a genuine edge.

▼ Show code

```
s1_data <- make_sensitivity_data(scenarios$s1)

s1_data$table |>
  kbl(
    align   = c("l", "r", "r", "r", "r", "r"),
    caption = "Sensitivity Table: Scenario 1 — 200 bets, $100,000 starting bankroll"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed", "responsive"),
    full_width = TRUE
  )
```

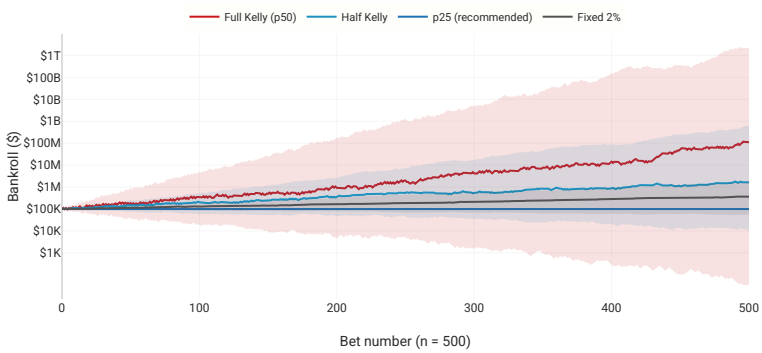
Sensitivity Table: Scenario 1 — 200 bets, $100,000 starting bankroll

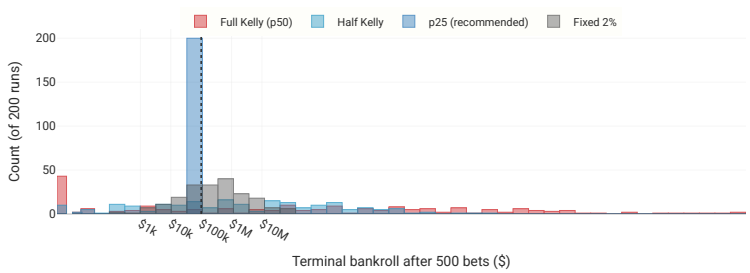
	Strategy	Stake	Bet ($100k)	Median final bankroll	Median drawdown	P(drawdown > 20%)
p10	p10	0.0%	$0	$100,000	0.0%	0.0%
p25	p25	0.0%	$0	$100,000	0.0%	0.0%
p50	p50	13.1%	$13,137	$544,447	78.2%	100.0%
p75	p75	29.3%	$29,268	$59,825	99.2%	100.0%
Fixed 2%	Fixed 2%	2.0%	$2,000	$161,617	17.0%	43.4%

Wealth trajectories

▼ Show code

```
make_trajectory_plots(scenarios$s1, s1_data$f_values)
```





> **Key insight.** With only 12 observations the p10 and p25 Kelly fractions are near zero — Kelly is correctly expressing that you know very little. A traditional point estimate calculation would recommend a confident stake around 16%. The Bayesian framework says: the data does not yet justify that confidence. The lower percentile stakes are zero not because the strategy is bad, but because the evidence is insufficient to distinguish a genuine edge from noise.

Scenario 2: The Transition

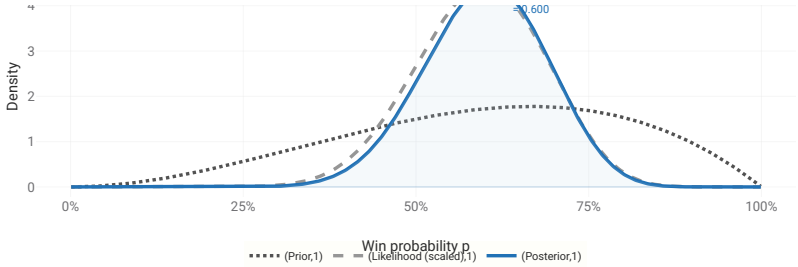
15 wins in 25 trials. Prior Beta(3, 2). Decimal odds 2.0.

More observations, a slightly more informative prior. The posterior mean has risen to 60% and — critically — the posterior has tightened enough that p25 of the Kelly distribution turns meaningfully positive. This is the transition point: the model now has enough evidence to recommend a cautious but non-zero stake at the conservative percentile.

Prior vs. posterior

▼ Show code

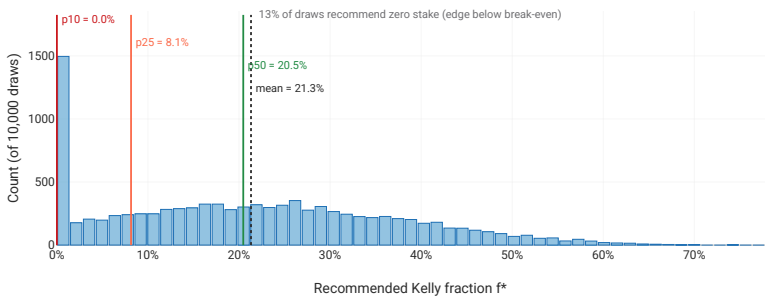
```
make_density_plot(scenarios$s2)
```



Kelly fraction distribution

▼ Show code

```
make_kelly_plot(scenarios$s2)
```



Staking policy

Understanding the sensitivity table (click to expand)

▼ Show code

```
s2_data <- make_sensitivity_data(scenarios$s2)

s2_data$table |>
  kbl(
    align   = c("l", "r", "r", "r", "r", "r"),
    caption = "Sensitivity Table: Scenario 2 — 200 bets, $100,000 starting bankroll"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed", "responsive"),
    full_width = TRUE
  )
```

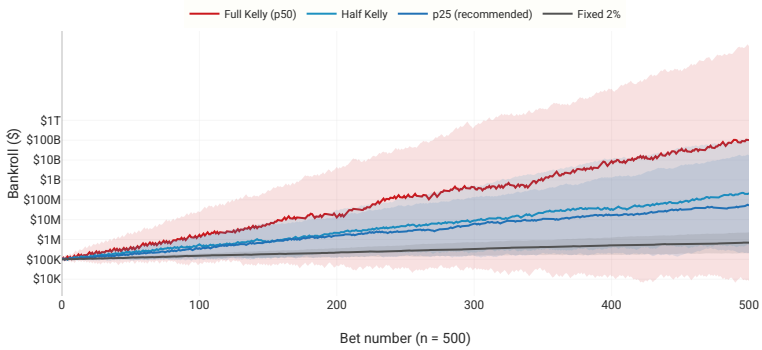
Sensitivity Table: Scenario 2 — 200 bets, $100,000 starting bankroll

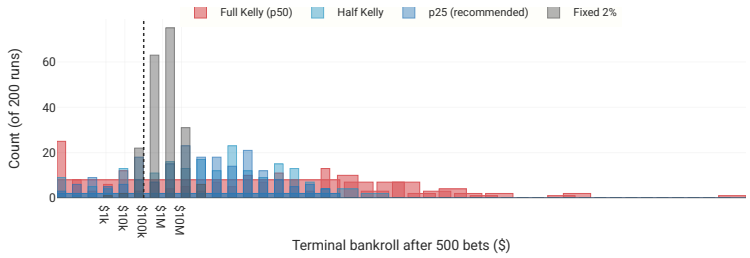
	Strategy	Stake	Bet ($100k)	Median final bankroll	Median drawdown	P(drawdown > 20%)
p10	p10	0.0%	$0	$100,000	0.0%	0.0%
p25	p25	8.1%	$8,127	$1,577,304	48.0%	99.0%
p50	p50	20.6%	$20,630	$8,490,830	87.2%	100.0%
p75	p75	32.1%	$32,132	$2,205,380	98.3%	100.0%
Fixed 2%	Fixed 2%	2.0%	$2,000	$213,849	13.5%	28.1%

Wealth trajectories

▼ Show code

```
make_trajectory_plots(scenarios$s2, s2_data$f_values)
```





> **Key insight.** The p25 stake has turned positive — the transition from "do not bet" to "cautiously deploy capital" is not a judgment call, it is a mathematical consequence of accumulating evidence. The overconfidence premium — the gap between p50 and p25 — remains large, reflecting that estimation uncertainty is still substantial. The fixed 2% strategy now looks conservative relative to the p25 recommendation, reversing the Scenario 1 picture.

Scenario 3: The Experienced Operator

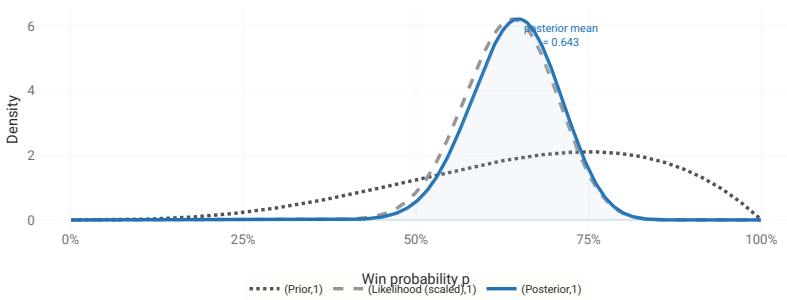
32 wins in 50 trials. Prior Beta(4, 2). Decimal odds 2.0.

Rich data and an informative prior. The posterior has tightened considerably — the 90% credible interval on the win probability has narrowed from the wide range of Scenario 1 to a much tighter band. As a consequence, p25 and p50 of the Kelly distribution have converged. The framework is granting permission to size up — not because the strategy changed, but because the evidence for it has grown.

Prior vs. posterior

▼ Show code

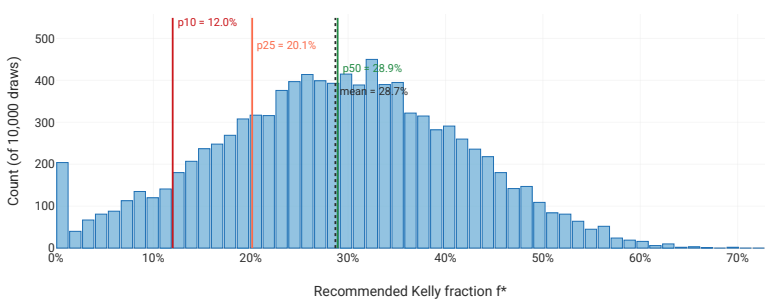
```
make_density_plot(scenarios$s3)
```



Kelly fraction distribution

▼ Show code

```
make_kelly_plot(scenarios$s3)
```



Staking policy

> **Understanding the sensitivity table (click to expand)**

▼ Show code

```
s3_data <- make_sensitivity_data(scenarios$s3)

s3_data$table |>
  kbl(
    align   = c("l", "r", "r", "r", "r", "r"),
    caption = "Sensitivity Table: Scenario 3 — 200 bets, $100,000 starting bankroll"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed", "responsive"),
```

```
  full_width = TRUE
)
```

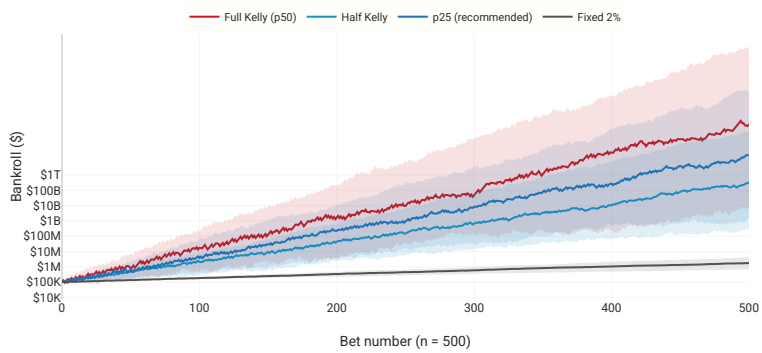
Sensitivity Table: Scenario 3 — 200 bets, $100,000 starting bankroll

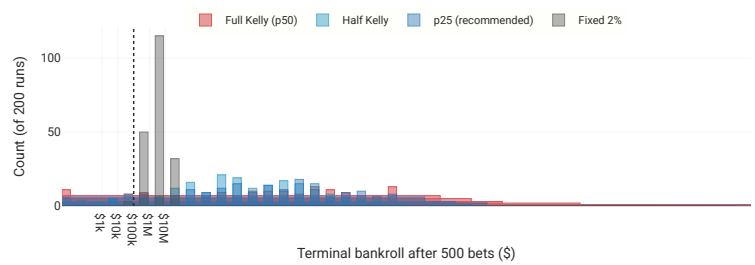
	Strategy	Stake	Bet ($100k)	Median final bankroll	Median drawdown	P(drawdown > 20%)
p10	p10	12.5%	$12,544	$30,733,163	57.0%	100.0%
p25	p25	20.2%	$20,184	$223,163,305	77.2%	100.0%
p50	p50	29.1%	$29,098	$507,422,382	92.5%	100.0%
p75	p75	37.3%	$37,277	$233,012,029	97.9%	100.0%
Fixed 2%	Fixed 2%	2.0%	$2,000	$306,530	11.5%	10.6%

Wealth trajectories

▼ Show code

```
make_trajectory_plots(scenarios$s3, s3_data$f_values)
```





> **Key insight.** The overconfidence premium has shrunk dramatically. With 50 observations the model trusts its own estimate enough that conservative and median sizing have converged. This is the self-correcting property of the Bayesian framework: as evidence accumulates, caution becomes permission. Note that this permission is earned — it cannot be manufactured by starting with a more aggressive prior.

Scenario 4: Optimism Meets Reality

7 wins in 12 trials. Prior Beta(2, 2). Decimal odds 2.0. True win rate: 52%.

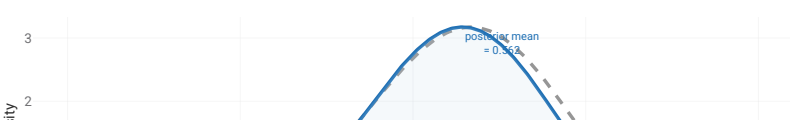
This scenario uses identical inputs to Scenario 1. The Kelly fractions recommended by the model are therefore identical. The difference is in the simulation: rather than drawing win outcomes from the posterior, we use a fixed true win probability of 52% — just above break-even, lower than the model's posterior mean of 56.3%. This represents the most common real-world failure mode: a model calibrated on in-sample data that overstates out-of-sample performance.

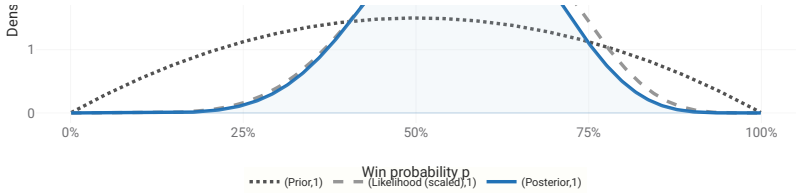
> **What changes in this scenario.** The prior, data, and Kelly fractions shown below are identical to Scenario 1. Only the wealth simulation changes: outcomes are drawn from a fixed true win rate of 52% rather than the posterior. The model does not know this — it still recommends stakes based on its 56.3% posterior mean estimate. The question is which staking strategy survives when the model is systematically optimistic.

Prior vs. posterior

▼ Show code

```
make_density_plot(scenarios$s4)
```

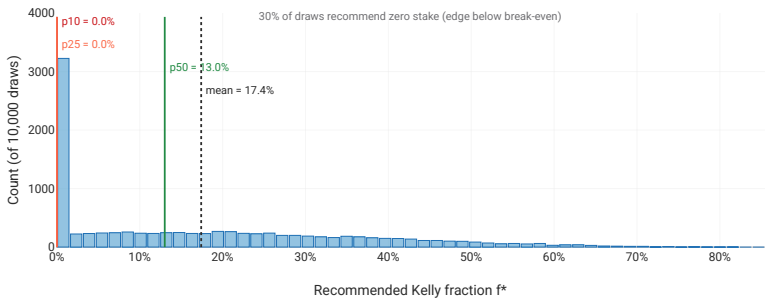




Kelly fraction distribution

▼ Show code

```
make_kelly_plot(scenarios$s4)
```



Staking policy

Understanding the sensitivity table (click to expand)

▼ Show code

```
s4_data <- make_sensitivity_data(scenarios$s4)

s4_data$table |>
  kbl(
    align   = c("l", "r", "r", "r", "r", "r"),
    caption = "Sensitivity Table: Scenario 4 — 200 bets at true win rate 52%, $100,000 starting bankr
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed", "responsive"),
    full_width = TRUE
  )
```

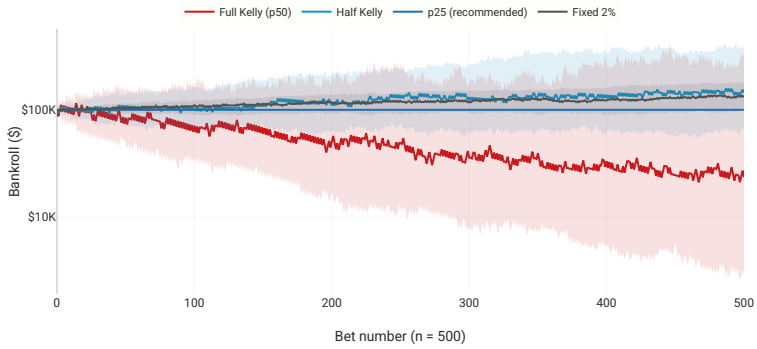
Sensitivity Table: Scenario 4 — 200 bets at true win rate 52%, $100,000 starting bankroll

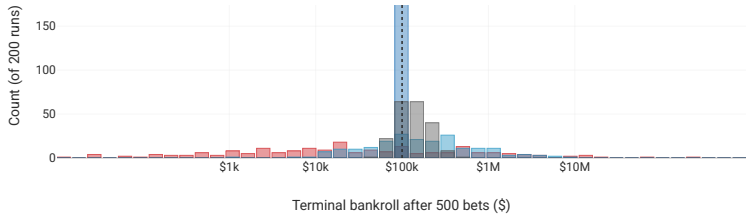
	Strategy	Stake	Bet ($100k)	Median final bankroll	Median drawdown	P(drawdown > 20%)
p10	p10	0.0%	$0	$100,000	0.0%	0.0%
p25	p25	0.0%	$0	$100,000	0.0%	0.0%
p50	p50	13.1%	$13,137	$50,466	90.5%	100.0%
p75	p75	29.3%	$29,268	$144	100.0%	100.0%
Fixed 2%	Fixed 2%	2.0%	$2,000	$112,751	23.4%	66.6%

Wealth trajectories

▼ Show code

```
make_trajectory_plots(scenarios$s4, s4_data$f_values)
```





> **Key insight.** This is the scenario practitioners most often skip. Full Kelly, sized for a 56% win rate, is severely punished by a 52% reality — the edge is real but the model has overstated it. The p25 stake, already near zero precisely because the posterior was wide and uncertain, suffers almost no damage. The conservative sizing that looked timid in Scenario 1 turns out to be robust protection against model optimism. Fixed 2% also survives, but at the cost of leaving growth on the table when the model is correct.

Comparative summary

The table below collects the key diagnostic numbers across all four scenarios in one place. Before reading it, here is a plain-English guide to each column so that the numbers tell a complete story rather than a technical one.

Scenario is simply the name we gave each case. Each scenario represents a different amount of evidence about a strategy's win rate.

Prior describes our starting belief before we observed any data. Beta(2, 2) is the most humble starting point — it says "I have almost no prior knowledge; the win rate could be anything between 0% and 100%." Beta(4, 2) says "I have some prior reason to believe the win rate is above 50%." Think of it as the difference between a first-time analyst with no track record and an experienced operator with years of comparable data.

Wins / Trials is the raw observed data. 7 wins in 12 trials means the strategy won 58% of the time in the sample. 32 wins in 50 trials also gives roughly a 64% observed win rate — but the second number is far more trustworthy because it is based on four times as much evidence.

Posterior mean is what the model believes the true win rate is after combining the prior with the observed data. Because our odds are 2.0 (even money), any win rate above 50% means a genuine edge. The higher the posterior mean, the stronger the estimated edge.

p25 Kelly is the recommended stake at the 25th percentile of the Kelly distribution. Think of it this way: imagine 10,000 different equally plausible values of the true win rate, each producing its own Kelly recommendation. The p25 Kelly is the stake that beats 25% of those recommendations and is beaten by 75% of them. It is the conservative choice — right if the true win rate is anywhere in the upper three-quarters of the uncertainty range.

p50 Kelly is the median Kelly recommendation across those same 10,000 draws. This is the closest equivalent to what a traditional point-estimate Kelly calculation would suggest. It represents the "most likely" recommended stake given the posterior.

Overconfidence premium is the gap between p50 and p25. This number answers the question: *how much are you over-staking if you use the median recommendation instead of the conservative one?* A large premium means the data is thin and the posterior is wide — your point estimate could easily be wrong and the penalty for being wrong is large. A small premium means the evidence has accumulated and the conservative and median recommendations have converged. It is the single most important number in this table for a risk manager deciding how much capital to deploy.

▼ Show code

```
summary_rows <- lapply(list(
  list(sc = scenarios$s1, dat = s1_data),
  list(sc = scenarios$s2, dat = s2_data),
  list(sc = scenarios$s3, dat = s3_data),
  list(sc = scenarios$s4, dat = s4_data)
), function(x) {
  sc  <- x$sc
  pp  <- posterior_params(sc)
  fv  <- x$dat$f_values
  pm  <- pp$alpha_post / (pp$alpha_post + pp$beta_post)
  set.seed(42)
  post_p  <- rbeta(5000, pp$alpha_post, pp$beta_post)
  f_draws <- kelly_fraction(post_p, sc$b_odds)
  f_sd    <- sd(f_draws)

  data.frame(
    Scenario              = paste0(sc$label, ": ", sc$title),
    `Prior`               = sprintf("Beta(%d, %d)", sc$alpha_prior, sc$beta_prior),
    `Wins / Trials`       = paste0(sc$wins, " / ", sc$n_trials),
    `Posterior mean`      = percent(pm, accuracy = 0.1),
    `p25 Kelly`           = percent(fv["p25"], accuracy = 0.1),
    `p50 Kelly`           = percent(fv["p50"], accuracy = 0.1),
    `Overconfidence premium` = percent(max(fv["p50"] - fv["p25"], 0), accuracy = 0.1),
    check.names = FALSE
  )
])

do.call(rbind, summary_rows) |>
  kbl(
    align   = c("l", "c", "c", "r", "r", "r", "r"),
    caption = "Comparative summary across four scenarios (odds = 2.0 throughout)"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed", "responsive"),
    full_width = TRUE
```

```
  ) |>
  column_spec(7, bold = TRUE)
```

Comparative summary across four scenarios (odds = 2.0 throughout)

	Scenario	Prior	Wins / Trials	Posterior mean	p25 Kelly	p50 Kelly	Overconfidence premium
p25	Scenario 1: The Danger Zone	Beta(2, 2)	7 / 12	56.2%	0.0%	**13.1%**	13.1%
p251	Scenario 2: The Transition	Beta(3, 2)	15 / 25	60.0%	8.1%	**20.6%**	12.5%
p252	Scenario 3: The Experienced Operator	Beta(4, 2)	32 / 50	64.3%	20.2%	**29.1%**	8.9%
p253	Scenario 4: Optimism Meets Reality	Beta(2, 2)	7 / 12	56.2%	0.0%	**13.1%**	13.1%

Reading across the rows, the story the table tells is straightforward. In Scenario 1, we have very little data (12 trials) and a humble prior. The posterior mean of 56.3% suggests a positive edge, but the overconfidence premium of roughly 12–15 percentage points means that if we stake at the median Kelly recommendation, we are implicitly betting on the precision of our own estimate. The conservative p25 recommendation is near zero — not because there is no edge, but because there is not yet enough evidence to be confident the edge is real.

By Scenario 2, we have more than doubled our observations to 25 trials and the prior is slightly more informative. The p25 recommendation has turned meaningfully positive. The model now has enough evidence to cautiously deploy capital at the conservative percentile. The overconfidence premium is still substantial, which means the median recommendation remains significantly above what the model considers well-supported.

Scenario 3 is the payoff for patience. With 50 trials and a stronger prior, the posterior has tightened considerably. The p25 and p50 recommendations have converged — the model is granting permission to size up because the evidence now supports it. The overconfidence premium has shrunk to its smallest value across all four scenarios. This is the self-correcting property of the framework in action: the more evidence you accumulate, the more the conservative and aggressive recommendations agree.

Scenario 4 uses the same inputs as Scenario 1, so its Kelly fractions are identical. But the simulation uses a true win rate of only 52% rather than the posterior mean. The table shows what the model recommends; the trajectory charts show what actually happens. The p25 Kelly stake, already near zero because the posterior was wide, essentially sits out the damage. Full Kelly does not.

Takeaways & Conclusion

Most practitioners learn the Kelly formula as a single equation that spits out a single number: stake this percentage of your bankroll. The formula is mathematically elegant and provably optimal — but optimal only under a condition that is almost never true in practice. That condition is that you know your edge precisely. You don't. Nobody does. Every win rate estimate comes from a finite sample of data, and finite samples are noisy. The Bayesian Kelly framework in this project is simply the Kelly formula with that noise made explicit.

Here's what the four scenarios teach us.

Lesson 1: A positive edge estimate is not a known edge

In Scenario 1, the strategy won 7 out of 12 times, giving a 58% win rate. The traditional Kelly formula suggests a confident 16% stake. But 12 observations are hardly enough. If you flipped a fair coin 12 times, you'd get 7 or more heads about 39% of the time just by chance. This means Scenario 1 data is mostly noise. A Bayesian analysis reveals that the win rate's distribution is very broad. This means many likely win rates are below 50%. Thus, Kelly correctly recommends a zero stake here.

The practical takeaway is clear. A backtest with 12 trades and a 58% win rate means: "The Kelly formula needs a known probability." We don't have that; we have a noisy sample. Show me what Kelly recommends across all plausible win rates before we discuss position sizing."

Lesson 2: Evidence allows for sizing up; not the other way around

A common instinct in managing portfolios is to start with a position and add as the trade goes well. The Bayesian Kelly framework flips that logic. You don't size up just because the trade is working. You size up when you have enough evidence that your edge is real.

Scenarios 1 through 3 show this evolution. The win rate estimate is similar in all three — around 58% to 64%. What changes is the sample size and prior quality. In Scenario 1, the p25 Kelly recommendation is near zero. By Scenario 3, it grows to a meaningful fraction of full Kelly. The strategy remains the same; only the evidence for it changes. The framework automatically turns that evidence into a position sizing recommendation. It does this without needing any judgment.

For risk or investment committees, the right question is not just, "What is the estimated win rate?" but "How many independent observations support that estimate, and how wide is the uncertainty around it?" A strategy with a 60% win rate over 10 trades is not the same as one with the same rate over 500 trades. The Bayesian framework treats them differently. Your risk processes should, too.

Lesson 3: The real risk is not losing bets — it's being wrong about your model

Scenario 4 is crucial and often overlooked. It asks: what if your model is right about the edge's direction but wrong about its size?

In Scenario 4, the model estimates a 56% win rate, while the reality is 52%. That 4-point gap seems small. On one bet, it's almost irrelevant. But over 500 bets, it can be disastrous for full Kelly. Why? Because Kelly is very sensitive to the edge estimate. When you stake at full Kelly for a 56% win rate but operate in a 52% world, you consistently over-stake every bet. This compounds the error. Over hundreds of bets, full Kelly can lead to near ruin despite a positive edge.

The p25 stake survives this not because it's smarter, but because it's smaller. It is conservatively sized due to the wide posterior, which indicates uncertainty about the estimate. This uncertainty turned out to be justified. The cautious sizing was not timidity; it was the right response to genuine uncertainty about the model's output.

In practice, the gap between in-sample and out-of-sample performance is common. Every strategy tested on historical data tends to overstate future performance. The key question is not if your model is optimistic — it probably is. The question is whether your position sizing reflects that optimism or ignores it. Full Kelly ignores it. The p25 Bayesian stake accounts for it.

Lesson 4: The overconfidence premium is the number that your board should see

When presenting a new strategy or allocation to a board or committee, you usually suggest one position size. This comes with a confidence interval or a value-at-risk figure. This framing is incomplete because it misses the biggest risk: the edge estimate's accuracy.

The overconfidence premium — the gap between the p50 and p25 Kelly recommendations — quantifies that risk. A large premium indicates thin data and a potentially flawed estimate. A small premium suggests reliable evidence. Presenting both numbers and naming the gap changes the talk. It shifts from "here is our recommendation" to "here is our recommendation, our confidence in the evidence, and the cost of being wrong." This method is clearer and more reliable.

The Bayesian Kelly framework doesn't change the strategy or the edge quality. It adjusts how much you admit to not knowing and how that affects position size. For institutions that care about both capital protection and growth, this distinction is key. It's the difference between a true risk management process and merely appearing to have one.

Session info

```
─ Session info ───────────────────────────────────────────────────────────────
 setting  value
 version  R version 4.5.2 (2025-10-31)
 os       macOS Tahoe 26.2
 system   aarch64, darwin20
 ui       X11
 language (EN)
 collate  en_US.UTF-8
 ctype    en_US.UTF-8
 tz       America/New_York
 date     2026-04-30
 pandoc   3.6.3 @ /Applications/RStudio.app/Contents/Resources/app/quarto/bin/tools/aarch64/ (via
rmarkdown)
 quarto   1.8.26 @ /usr/local/bin/quarto

─ Packages ───────────────────────────────────────────────────────────────────
 package      * version date (UTC) lib source
 cli            3.6.5   2025-04-23 [1] CRAN (R 4.5.0)
 crosstalk      1.2.2   2025-08-26 [1] CRAN (R 4.5.0)
 data.table     1.17.8  2025-07-10 [1] CRAN (R 4.5.0)
 digest         0.6.39  2025-11-19 [1] CRAN (R 4.5.2)
 dplyr        * 1.1.4   2023-11-17 [1] CRAN (R 4.5.0)
 evaluate       1.0.5   2025-08-27 [1] CRAN (R 4.5.0)
 farver         2.1.2   2024-05-13 [1] CRAN (R 4.5.0)
 fastmap        1.2.0   2024-05-15 [1] CRAN (R 4.5.0)
 generics       0.1.4   2025-05-09 [1] CRAN (R 4.5.0)
 ggplot2      * 4.0.2   2026-02-03 [1] CRAN (R 4.5.2)
 glue           1.8.0   2024-09-30 [1] CRAN (R 4.5.0)
 gtable         0.3.6   2024-10-25 [1] CRAN (R 4.5.0)
 htmltools    * 0.5.8.1 2024-04-04 [1] CRAN (R 4.5.0)
 htmlwidgets    1.6.4   2023-12-06 [1] CRAN (R 4.5.0)
 httr           1.4.7   2023-08-15 [1] CRAN (R 4.5.0)
 jsonlite       2.0.0   2025-03-27 [1] CRAN (R 4.5.0)
 kableExtra   * 1.4.0   2024-01-24 [1] CRAN (R 4.5.0)
 knitr        * 1.50    2025-03-16 [1] CRAN (R 4.5.0)
 labeling       0.4.3   2023-08-29 [1] CRAN (R 4.5.0)
 lazyeval       0.2.2   2019-03-15 [1] CRAN (R 4.5.0)
 lifecycle      1.0.5   2026-01-08 [1] CRAN (R 4.5.2)
 magrittr       2.0.4   2025-09-12 [1] CRAN (R 4.5.0)
 pillar         1.11.1  2025-09-17 [1] CRAN (R 4.5.0)
 pkgconfig      2.0.3   2019-09-22 [1] CRAN (R 4.5.0)
 plotly       * 4.11.0  2025-06-19 [1] CRAN (R 4.5.0)
 purrr          1.2.0   2025-11-04 [1] CRAN (R 4.5.0)
 R6             2.6.1   2025-02-15 [1] CRAN (R 4.5.0)
 RColorBrewer   1.1-3   2022-04-03 [1] CRAN (R 4.5.0)
 rlang          1.1.7   2026-01-09 [1] CRAN (R 4.5.2)
 rmarkdown      2.30    2025-09-28 [1] CRAN (R 4.5.0)
 rstudioapi     0.17.1  2024-10-22 [1] CRAN (R 4.5.0)
 S7             0.2.1   2025-11-14 [1] CRAN (R 4.5.2)
 scales       * 1.4.0   2025-04-24 [1] CRAN (R 4.5.0)
 sessioninfo  * 1.2.3   2025-02-05 [1] CRAN (R 4.5.0)
 stringi        1.8.7   2025-03-27 [1] CRAN (R 4.5.0)
 stringr        1.6.0   2025-11-04 [1] CRAN (R 4.5.0)
 svglite        2.2.2   2025-10-21 [1] CRAN (R 4.5.0)
 systemfonts    1.3.1   2025-10-01 [1] CRAN (R 4.5.0)
 textshaping    1.0.4   2025-10-10 [1] CRAN (R 4.5.0)
 tibble         3.3.0   2025-06-08 [1] CRAN (R 4.5.0)
 tidyr        * 1.3.1   2024-01-24 [1] CRAN (R 4.5.0)
 tidyselect     1.2.1   2024-03-11 [1] CRAN (R 4.5.0)
 vctrs          0.7.1   2026-01-23 [1] CRAN (R 4.5.2)
 viridisLite    0.4.3   2026-02-04 [1] CRAN (R 4.5.2)
 withr          3.0.2   2024-10-28 [1] CRAN (R 4.5.0)
 xfun           0.54    2025-10-30 [1] CRAN (R 4.5.0)
 xml2           1.4.1   2025-10-27 [1] CRAN (R 4.5.0)
 yaml           2.3.10  2024-07-26 [1] CRAN (R 4.5.0)

 [1] /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library
 * ── Packages attached to the search path.

──────────────────────────────────────────────────────────────────────────────
```

Rendered with Quarto · Packages: `dplyr htmltools kableExtra knitr plotly scales sessioninfo tidyverse`